

Agentic RAG

LangGraph

LLM

Enterprise AI

Python

# How I Built an Enterprise Knowledge Assistant Using Agentic RAG and LangGraph

A practical walkthrough of designing a multi-agent AI system that retrieves, evaluates, and grounds its answers in real enterprise documents.

---

By Thusharkanth Loganathan · AI/ML Engineer · May 2026 · 12 min read

---

## 1. Opening

---

Have you ever wondered how much time is wasted inside enterprises just searching for information?

In most organizations, employees constantly switch between HR portals, onboarding documents, internal SOPs, and chat messages just to find simple answers. Something as basic as

*‘How do I report a security incident?’*

can easily turn into a 10–15 minute search or a message chain across multiple teams.

Now imagine a system where you simply ask a question — and within seconds, you receive a precise, context-aware answer backed by actual internal documents.

Not just an answer generated by a language model... but one that is grounded, verified, and traceable.

That idea led me to build an Enterprise Knowledge Assistant using Agentic RAG and LangGraph.

In this article, I'll walk through how I designed the system, why I moved beyond traditional RAG, and what challenges I faced while building a multi-agent AI workflow.

## 2. The Problem — Why Traditional Approaches Fail

---

When I started thinking about this system, the initial idea seemed simple:

“Just use a RAG pipeline over enterprise documents.”

But in reality, enterprise knowledge systems are messy.

**Documents are:**

- scattered across folders
- written in different formats
- partially outdated
- often context-dependent

And most importantly — not every retrieved chunk is useful.

A simple RAG system blindly retrieves top-k chunks and sends them to the LLM. This leads to:

- irrelevant answers
- hallucinations
- weak grounding
- inconsistent responses

I realized early that retrieval alone is not enough. The system also needs to decide whether the retrieved context is actually sufficient before generating an answer.

That realization changed the architecture completely.

---

## 3. The Idea — Moving Toward Agentic RAG

---

Instead of building a linear pipeline:

**Retrieve → Generate**

I moved toward a more intelligent flow:

**Retrieve → Evaluate → Decide → Generate**

This introduces a key concept: the system should reason about its own context before answering.

That's where the idea of Agentic RAG came in.

Instead of one monolithic pipeline, I designed multiple specialized components (agents), each responsible for a specific decision.

This also naturally aligned with LangGraph, which allows state-based orchestration of workflows.

## 4. System Architecture — High-Level Design

The system is divided into four main layers:

UI Layer — Streamlit frontend for query input and response display

Orchestration Layer — LangGraph for state-based agent workflow management

Retrieval Layer — ChromaDB vector store and Ollama embeddings

Generation Layer — Google Gemini 1.5 Flash for evaluation and answer generation

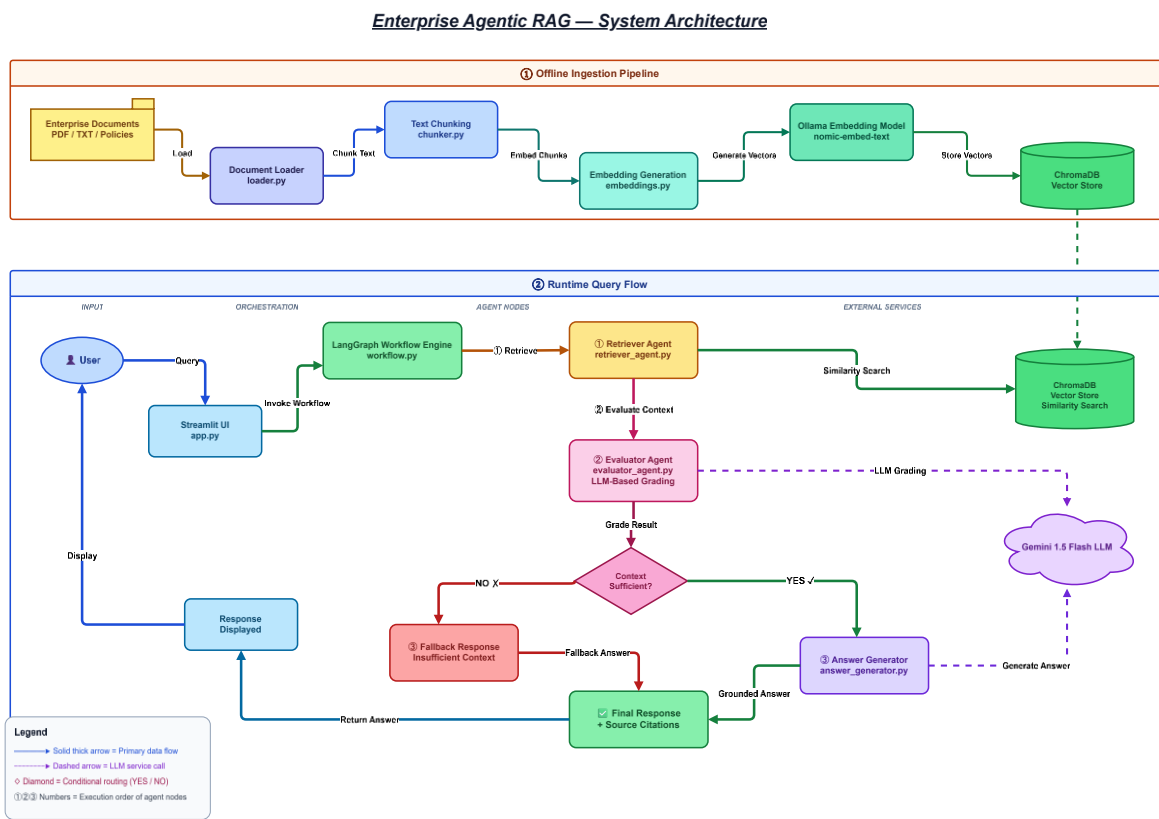


Figure 1 — Full system architecture showing the design

What makes this architecture powerful is that each layer is independent and replaceable. This modularity is what makes it 'enterprise-like' rather than just a script.

## 5. Building the Ingestion Pipeline

### Turning Documents into Knowledge

Before answering any question, the system needs structured knowledge. So I built a document ingestion pipeline.

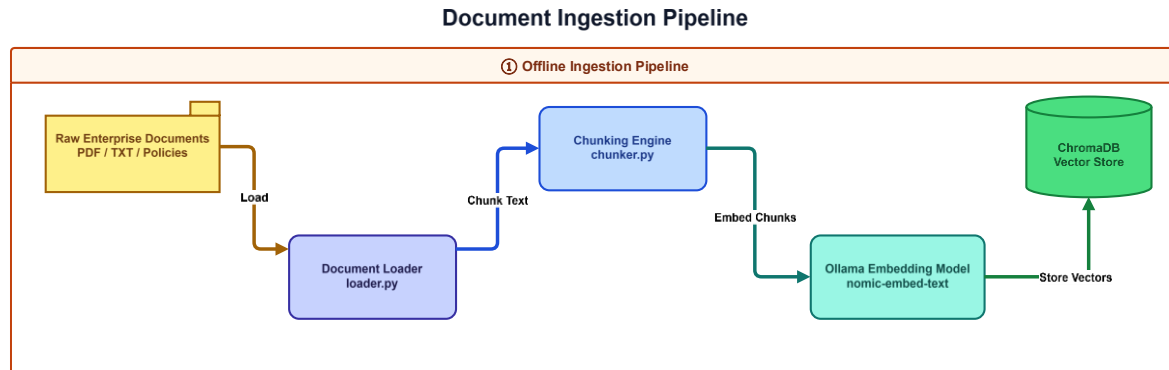


Figure 2 — Document ingestion flow: Load → Chunk → Embed → Store

#### Step 1 — Loading Documents

All enterprise documents — HR policies, cybersecurity guidelines, onboarding guides, and SOPs — are loaded from local storage using LangChain's DirectoryLoader and TextLoader.

#### Step 2 — Chunking

Instead of feeding entire documents to the LLM, I split them into smaller, focused chunks. This is critical because:

- LLMs perform better with focused, relevant context
- retrieval becomes more precise at the chunk level
- irrelevant noise is reduced, improving answer quality

#### Step 3 — Embedding Generation

Each chunk is converted into a vector representation using Ollama embeddings (nomic-embed-text). I chose local embeddings because:

- no external API cost
- full control over the embedding model
- easy experimentation and iteration

#### Step 4 — Vector Storage

All embeddings are stored in ChromaDB, persisted locally at `vectorstore/chroma_db/`. At this point, the system has a structured, searchable memory of enterprise knowledge.

## 6. Why I Used Agentic RAG Instead of Traditional RAG

Traditional RAG is simple — and in enterprise systems, that simplicity becomes a limitation.

| Traditional RAG                 | Agentic RAG                             |
|---------------------------------|-----------------------------------------|
| retrieve → generate             | retrieve → evaluate → decide → generate |
| blindly trusts retrieved chunks | evaluates context before answering      |
| no quality check on context     | LLM-based context quality assessment    |
| hallucinations remain possible  | falls back gracefully when uncertain    |
| fixed, linear execution         | conditional, state-driven execution     |

Introducing an evaluator step transforms the pipeline into a reasoning system. This small addition significantly improves reliability across all query types.

## 7. LangGraph Workflow — The Core Engine

LangGraph acts as the orchestration layer of the system. The workflow is state-based, meaning each step receives a typed state object, transforms it, and passes it to the next node.

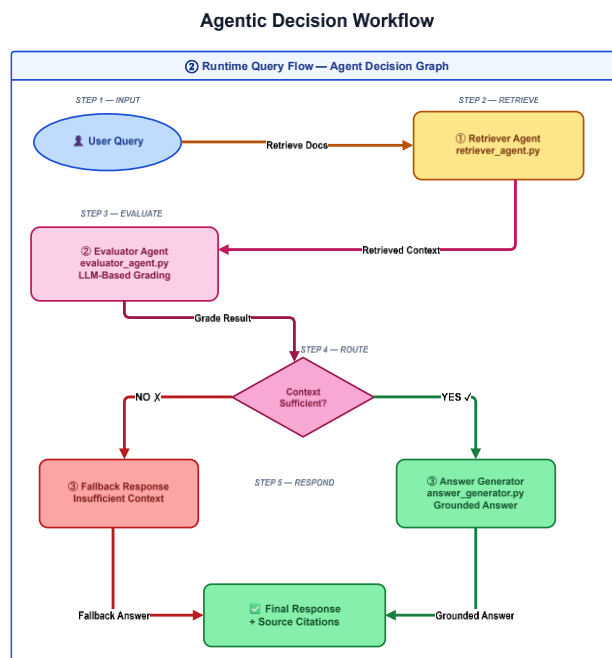


Figure 3 — LangGraph state machine: Retriever → Evaluator → Router → Generator / Fallback

### Step 1 — Retrieval Agent

Fetches relevant document chunks from ChromaDB using semantic similarity search. The user's query is embedded using Ollama and compared against stored vectors to retrieve the top-k most relevant chunks.

### Step 2 — Evaluator Agent

This is where the system becomes intelligent. Instead of blindly trusting retrieval, the evaluator checks:

**Is the retrieved context sufficient to answer the question?**

Initially, I used a simple length-based heuristic. But later, I upgraded to an LLM-based evaluator using Gemini. Now the system can actually reason about context quality — assessing relevance, completeness, and answerability.

### Step 3 — Conditional Routing

Based on evaluation, LangGraph conditionally routes execution:

- If sufficient → generate a grounded answer
- If insufficient → return a graceful fallback response

### Step 4 — Answer Generation

The final agent generates a structured response using the query, retrieved context, and a carefully engineered prompt. It also appends source document references for full transparency.

---

## 8. The Evaluator Agent — A Key Design Decision

---

One of the most important improvements in this project was replacing rule-based evaluation with an LLM-based evaluator.

### The original approach:

```
if len(context) < threshold:  
    return 'insufficient'
```

But this approach is fundamentally weak because:

- **length ≠ quality** — a long but irrelevant chunk still passes
- short but precise answers unfairly fail the threshold
- it cannot reason about semantic relevance

So I switched to a prompt-based evaluator using Gemini. Now the system evaluates three dimensions:

**Relevance** — Does the retrieved context actually relate to the query?

Completeness — Is there enough information to construct a full answer?

Answerability — Can the question be answered accurately from the context?

This change significantly improved response accuracy across all query types.

---

## 9. Source Citations — Building Trust in AI Responses

---

One major challenge with LLM systems is trust. Without knowing where an answer came from, it is difficult to verify or audit.

To solve this, I added source tracking from retrieved chunks. Every response now includes the document name, origin file, and a traceable source reference.

Every answer is not just generated — it is grounded and cited.

This ensures transparency, auditability, and enterprise readiness. Teams can verify every answer against the source document it was derived from.

---

## 10. Challenges I Faced

---

This project wasn't straightforward. Here are the four main challenges I encountered:

### 1. LLM Response Parsing Issues

Gemini sometimes returned unexpected formats — lists instead of strings, or structured JSON where plain text was expected — which broke evaluation logic. I added defensive parsing with fallback handling to manage inconsistent outputs.

### 2. Retrieval Quality Tuning

Finding the right balance of chunk size, chunk overlap, and embedding model required significant experimentation. The wrong configuration produced either too-broad or too-narrow retrievals, both of which harmed answer quality.

### 3. LangGraph Debugging

Designing state transitions between nodes required careful attention to graph flow logic. Debugging a stateful graph is fundamentally different from debugging a sequential script — tools like LangSmith helped significantly.

### 4. Local Embedding Setup

Running Ollama embeddings required correct model configuration, environment variable setup, and ensuring the local model was properly downloaded and serving before ingestion. This added friction to initial setup.

## 11. Final Results

The system successfully delivers on its core goals across all major query types.

|                                                                 |
|-----------------------------------------------------------------|
| Retrieves relevant enterprise knowledge from internal documents |
| Evaluates context quality using LLM-based reasoning             |
| Generates grounded, accurate responses with source citations    |
| Falls back gracefully when context is insufficient              |
| Prevents hallucinations through retrieval augmentation          |

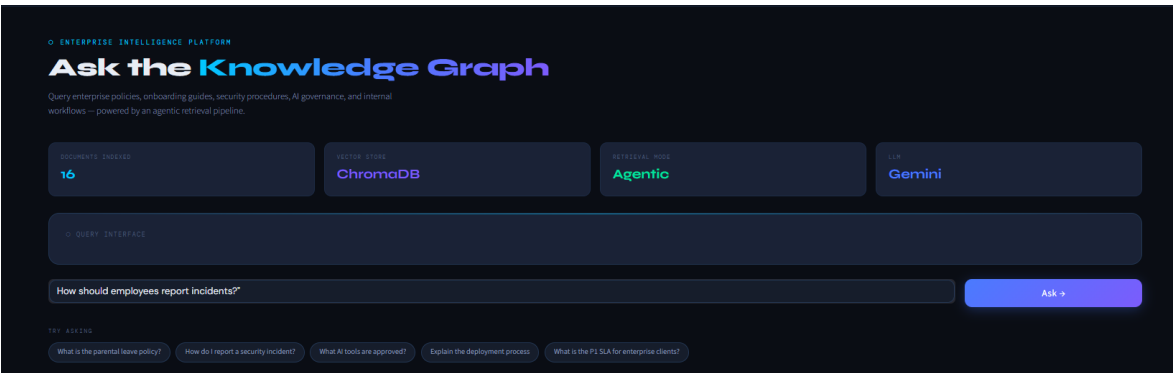
### Example Query

Query: "How should employees report security incidents?"

The system retrieved relevant chunks from the cybersecurity guidelines and incident management workflow documents, evaluated them as sufficient, generated a structured policy-based response, and appended source citations:

Sources:

- 09\_incident\_management\_workflow.txt
- 06\_cybersecurity\_guidelines.txt





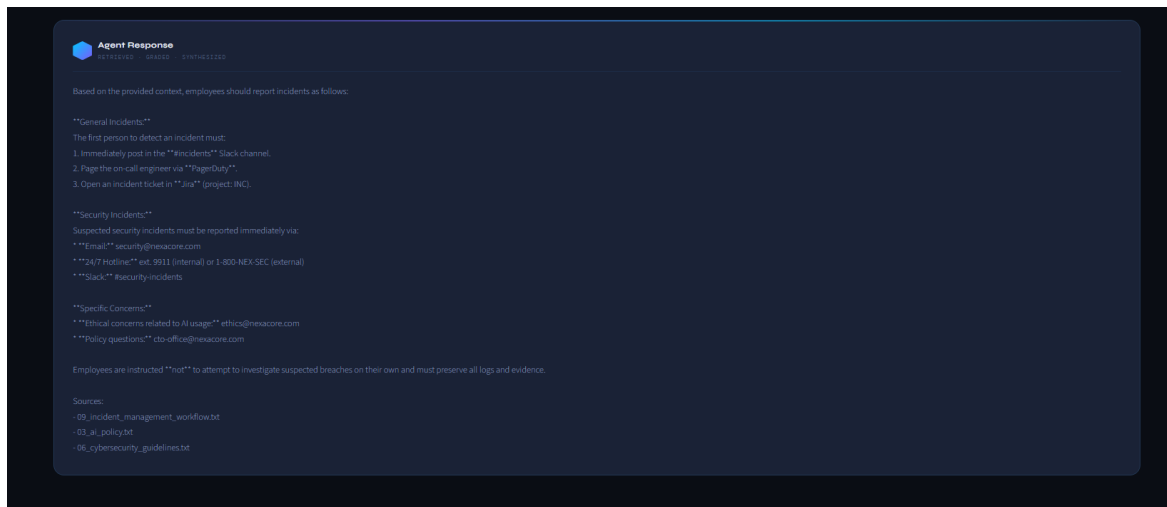


Figure 4 — Streamlit UI showing a live query with grounded response and source citations

## 12. What I Learned

This project reshaped how I think about building AI systems. A few key takeaways:

- **Real RAG systems are messier than tutorials suggest.** Evaluation layers, retrieval tuning, and fallback handling are as important as the retrieval itself.
- **Evaluation layers matter.** A system that knows when not to answer is significantly more trustworthy than one that always attempts a response.
- **Retrieval quality directly impacts LLM output.** Garbage in, garbage out — no amount of prompt engineering compensates for poor retrieval.
- **Orchestration frameworks unlock modularity.** LangGraph enabled clean separation of concerns that would have been tangled in a monolithic script.
- **Building AI systems is system design.** The model is only one component. The architecture around it determines reliability, maintainability, and enterprise-readiness.

## 13. Future Improvements

If I extend this system, the highest-value additions would be:

- Hybrid search combining BM25 keyword retrieval with dense vector search for improved recall
- Multi-agent collaboration with specialist agents for HR, security, compliance, and IT domains
- Memory-based conversation layer enabling multi-turn dialogue with session context retention
- Web search fallback agent for queries that fall outside the enterprise document corpus

- Authentication and role-based access control governing document-level permissions
  - Production deployment pipeline with containerisation and monitoring
- 

## 14. Conclusion

---

This project was my introduction to building real-world AI systems beyond simple chatbots. Instead of just generating responses, the system retrieves knowledge, evaluates context, makes decisions, and generates grounded outputs.

That shift — from a static RAG pipeline to an agentic reasoning system — is what makes this project meaningful.

The architecture demonstrates that enterprise AI is not just about choosing the right model. It is about designing systems that are reliable, transparent, modular, and trustworthy — capable of knowing when they know enough, and when they do not.

---

Agentic RAG

LangGraph

LLM

Enterprise AI

Python

Written by Thusharkanth Loganathan · AI/ML Engineer Intern · May 2026